

Resolución: Planificación Óptima de Paradas de Recarga para Coche Eléctrico

Este documento presenta la resolución detallada del problema de encontrar el número mínimo de paradas de recarga necesarias para viajar entre dos ciudades en un coche eléctrico con una autonomía máxima de x kilómetros, utilizando un enfoque de **Algoritmos Voraces** (Greedy).

1. Estrategia Algorítmica Elegida

Para este problema elegimos una **Estrategia Voraz** (Greedy).

Justificación de la Elección

La propiedad que queremos optimizar es el número total de paradas (minimizarlo). Dado que cada parada de recarga vuelve a llenar completamente la batería del coche dándole de nuevo la autonomía máxima completa (x kilómetros), no existe ninguna ventaja en parar antes de lo estrictamente necesario.

La estrategia óptima consiste en **conducir lo máximo posible en cada etapa sin quedarse sin batería**, y recargar en la última estación de carga que sea alcanzable antes de que la autonomía se agote por completo.

- **Elección voraz:** En cada paso, elegimos avanzar hasta la estación de carga más lejana que se encuentre a una distancia menor o igual a x de nuestra posición actual.

2. Diseño del Algoritmo Eficiente

Modelado de la Entrada

Representamos el trayecto mediante un arreglo de distancias:

- $d[0 \dots n]$: Donde $d[i]$ es la distancia desde el punto de origen (posición 0) hasta la estación de carga i .
- El origen es la posición $d[0] = 0$.
- El destino final es la última posición $d[n]$.
- x : La autonomía máxima del vehículo con una carga completa (asumimos que al salir del origen el coche está completamente cargado).

Consideración de Viabilidad

Si en algún punto del camino la distancia entre dos estaciones consecutivas es estrictamente mayor que la autonomía del coche ($d[i] - d[i - 1] > x$), es imposible realizar el viaje y el algoritmo debe reportarlo.

Algoritmo en Pseudocódigo

```

Funcion PlanificarParadasRecarga(d, n, x)
Inicio
    paradas_elegidas <- Lista vacía // Guardará los índices de las estaciones don
    posicion_actual <- 0           // Empezamos en el origen (índice 0)
    i <- 1                         // Intentamos avanzar hacia la estación i

    Mientras posicion_actual < n Hacer
        // 1. Encontrar la estación más lejana alcanzable desde la posición actua
        ultima_alcanzable <- posicion_actual

        Mientras i <= n y (d[i] - d[posicion_actual]) <= x Hacer
            ultima_alcanzable <- i
            i <- i + 1
        FinMientras

        // 2. Control de Viabilidad: Si no pudimos avanzar nada en esta iteración
        Si ultima_alcanzable = posicion_actual Entonces
            Escribir("Error: No es posible completar el recorrido. Autonomía insu
            Retornar Lista vacía
        FinSi

        // 3. Si la última estación alcanzable es el destino final, el viaje ha t
        Si ultima_alcanzable = n Entonces
            posicion_actual <- n
        Sino
            // De lo contrario, debemos parar obligatoriamente en esta última est
            Añadir ultima_alcanzable a paradas_elegidas
            posicion_actual <- ultima_alcanzable
            // No incrementamos 'i' aquí porque ya está apuntando a la siguiente
        FinSi
    FinMientras

    Retornar paradas_elegidas
FinFuncion

```

3. Coste del Algoritmo

Complejidad Temporal: $O(n)$

A pesar de tener dos bucles anidados (*Mientras*), el algoritmo es extremadamente eficiente y tiene un coste lineal:

Complejidad Espacial: $O(1)$ adicional

El algoritmo no requiere estructuras de datos auxiliares complejas que escalen con el tamaño del problema.